

FAÇA UM ÚNICO SCRIPT QUE EXECUTE TODAS AS TAREFAS ABAIXO

TAREFA 1 – LEITURA E ANÁLISE DAS ESTRUTURAS MATRICIAIS

- a) Faça a leitura das imagens `cameraman.tif` e `morangos.tif` para variáveis `img_1` e `img_2`, respectivamente, no Python e no Matlab. Tanto no OpenCV-Python, quanto no Matlab, isso pode ser feito por meio do comando `imread()`. No OpenCV-Python, utilize o parâmetro `cv2.IMREAD_UNCHANGED`, de forma a carregar a imagem tal como ela se encontra armazenada na memória;
- b) Em seguida, exiba as imagens, bem como suas dimensões. Para exibir a imagem, utilize a função `imshow()`, tanto no Python como no Matlab. No Python, esse comando existe tanto para as bibliotecas OpenCV-Python, `cv2`, quanto para a Matplotlib, `matplotlib.pyplot`. Na Matplotlib, é necessário utilizar o parâmetro `cmap='gray'`, caso a imagem a ser exibida esteja em tons de cinza. Além disso, ao exibir uma imagem ou gráfico, você pode criar uma janela específica para exibi-la, inclusive dando um nome a essa janela. Para isso, no Matplotlib e Matlab, utilize o comando `figure()`. Já no OpenCV-Python, você pode utilizar a função `cv2.namedWindow()`. Isso deve ser feito antes do comando `imshow()`, nos dois ambientes de programação. Por fim, no Python e Matlab, respectivamente, o cômputo das dimensões de uma imagem pode ser feito, por exemplo, como `img_1.shape` e `size(img_1)`, respectivamente. O método `shape` é aplicado a matrizes da biblioteca NumPy. As imagens carregadas por meio do OpenCV-Python estão também habilitadas a utilizar esse método. Lembre-se que, no Python, as informações devem ser exibidas por meio da função `print()`, enquanto no Matlab basta não utilizar ; ao final do comando. Por fim, compare a informação visual das cores das imagens com as dimensões apresentadas;

OBS: No IPython e no Matlab, o comando `whos` ajuda a verificar todas as variáveis já criadas no `workspace`. Note que `img_2` tem três dimensões, do tipo `NxMx3`, pois possui 3 cores (RGB). No Matlab, não coloque ; ao final do comando.

TAREFA 2 - SEPARANDO AS COMPONENTES (MATRIZES) DE CORES RGB

- a) Use a sintaxe `img_2[:, :, 2]` para Python e `img_2(:, :, 1)` para Matlab, de forma a armazenar, na variável `verm_1`, o componente/canal vermelho contido na imagem `img_2`;
- b) Exiba a imagem `verm_2`, dando a janela de exibição o nome “Morangos - Vermelho”;
- c) Separe agora também os canais verde e azul, da imagem `img_2`, nas variáveis `verde_2` e `azul_2`;

- d) Utilize agora o comando `dstack()`, por meio da biblioteca NumPy do Python, e `cat()`, no Matlab, para concatenar as três componentes de cores, `verm_2`, `verde_2` e `azul_2`. Armazene o resultado dessa operação na variável `img_rgb`. Lembre-se que é necessário importar a biblioteca NumPy para utilizar suas funções;
- e) Exiba a imagem `img_rgb`, dando a janela de exibição o título “Morangos - RGB”.

TAREFA 3 - CONVERTENDO UMA IMAGEM COLORIDA PARA:

TONS DE CINZA

- a) Converta a imagem `img_2` para tons de cinza. No Python, isso pode ser feito por meio do comando `cv2.cvtColor(img_2, cv2.COLOR_BGR2GRAY)` da biblioteca OpenCV-Python. Observe que se a imagem foi lida por meio do Matplotlib, e não pelo OpenCV-Python, o parâmetro empregado deverá ser `cv2.COLOR_RGB2GRAY`. No Matlab, essa conversão pode ser realizada por meio do comando `rgb2gray()`. Armazene o resultado na variável `img_cinza`;
- b) Exiba a imagem `img_cinza`, dando a janela de exibição da imagem o título “Morangos - Cinza”;
- c) Calcule a média das imagens `verm_2`, `verde_2` e `azul_2` e armazene na imagem `img_cinza_2`. Exiba a imagem `img_cinza_2`, dando a janela de exibição da imagem o título “Morangos - Cinza 2”. Entendeu como se converte uma imagem de RGB para tons de cinza?

OBS: Neste último item, antes de exibir a imagem, pode ser importante observar o tipo de dados da imagem. Se ela for do tipo `float16` ou `float32`, no Python, ou `double`, no Matlab, talvez seja necessário convertê-la para `uint8`. No Python, para saber o tipo de dados de uma variável, basta utilizar a função `type()`. No Matlab, pode-se empregar o comando `whos`.

HSV

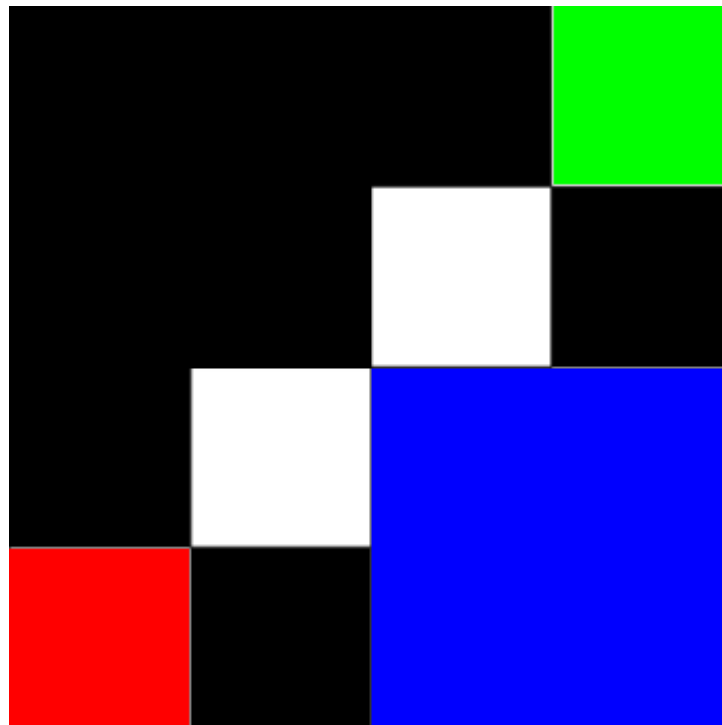
- a) Converta a imagem `img_2` para HSV. No Python, isso pode ser feito por meio dos parâmetros `cv2.COLOR_BGR2HSV` e `cv2.COLOR_RGB2HSV`, considerando a carga da imagem por meio das bibliotecas OpenCV-Python e Matplotlib, respectivamente. No Matlab, essa conversão pode ser feita por meio da função `rgb2hsv()`. Armazene o resultado na variável `img_hsv()`;
- b) Exiba a imagem `img_hsv`, dando a janela de exibição da imagem o título “Morangos - HSV”.

BINÁRIA

- d) Converta a imagem `img_1` para uma imagem binária. No Python, você deverá utilizar a função `cv2.threshold()`. No Matlab, o comando a ser utilizado é o `im2bw()`. Utilize como limiar o valor 128, armazenando o resultado na imagem `img_bin`;
- e) Exiba a imagem `img_bin`, dando a janela de exibição da imagem o título “Morangos – Binário”.

TAREFA 4 – PROCESSANDO UMA IMAGEM:

A imagem `xadrez.png` possui 361x361 pixels. Pede-se: gerar uma imagem, a partir da imagem `xadrez.png` e utilizando uma rotina em Matlab ou Python, tal qual a apresentada abaixo:



TAREFA 5 – PROCESSANDO UM VÍDEO:

Capturar quadros com uma webcam, em Matlab ou Python, converter cada quadro para o espaço de cores HSV e exibir o resultado no monitor. Os códigos-fonte `webcam_python.py` e `webcam_matlab.m` realizam a captura e exibição de imagens por meio de uma webcam conectada a um computador. Caso você não possua uma webcam, você pode realizar esse mesmo processo para um arquivo de vídeo, que esteja armazenado no disco do computador. Nesse caso, os códigos-fonte `video_file_matlab.m` e `video_file_python.py` realizam a captura e exibição dos frames de um vídeo. O vídeo `tracking_colors.mp4` é fornecido como amostra.